

HDFE controls in `did_multiplegt_dyn`

A fast path that bypasses the per-control loop
for dummy-basis controls like `ac_uq_id#monthyear`

1 What you are trying to do, and why it is slow today

You want to control for `ac_uq_id#monthyear` on the `data_test.csv` panel. `ac_uq_id` is a coarser group attribute (time-invariant at the group level), `monthyear` is the time index, and `ac_uq_id#monthyear` produces one dummy per (cluster, month) cell. On the size of your data this means **12,000+ dummies** — generated, first-differenced, residualized, fed to `matrix accum`, and then to a per-baseline `invsym` of a 12000×12000 matrix.

The existing path treats every control symmetrically:

- Stage 1, L902–941: generate one ΔX_k and one $\widetilde{\Delta X}_k$ per control.
- Stage 1, L1018: `matrix accum` on $K + 1$ residualized variables — a matrix of size $K \times K$ when $K = 12000$.
- Stage 1, L1034: `invsym(didmgt_XX)` — $O(K^3)$ in $K = 12000$ is roughly 10^{12} flops per baseline cell.
- Stage 2, L4679: K `replace` commands per horizon ℓ .
- Stage 3, L4905–4954: a nested K^2 loop on $K = 12000$ is $\sim 10^8$ iterations per horizon per baseline.

This is the wrong tool for dummy-basis controls. The right tool is fixed-effect absorption, and it gives the *same estimator*.

2 The math fact that powers the fast path: FWL

Take any partition of your dataset’s columns into “numeric controls” X and “dummy controls” Z (where Z are the indicators of the cells `ac_uq_id` $\times$ `monthyear`). The OLS of ΔY on ΔX and ΔZ gives the same numeric-controls slope $\hat{\theta}_d$ as the OLS of ΔY on ΔX after *partialling out the column space of ΔZ from both sides*:

$$\underbrace{\begin{bmatrix} \Delta X & \Delta Z \end{bmatrix}}_{=: \mathbf{X}_{\text{full}}}^\top \mathbf{W} \mathbf{X}_{\text{full}} \begin{bmatrix} \hat{\theta}_d \\ \hat{\phi}_d \end{bmatrix} = \mathbf{X}_{\text{full}}^\top \mathbf{W} \Delta Y \quad \Longleftrightarrow \quad \hat{\theta}_d = ((M_Z^W \Delta X)^\top \mathbf{W} (M_Z^W \Delta X))^{-1} (M_Z^W \Delta X)^\top \mathbf{W} M_Z^W \Delta Y, \quad (1)$$

where $M_Z^W = \mathbf{I} - P_Z^W$ is the (weighted) annihilator of ΔZ . Frisch–Waugh–Lovell, exactly as in any textbook.

Reading. The estimator we want *never has to materialize* the 12,000 columns of ΔZ . We only need their *projection action*: “demean by the column space of ΔZ .” For $Z = \mathbb{K}\{a_g = a_0\} \cdot \mathbb{K}\{t = t_0\}$, that action is computable in $O(N)$ per iteration of the alternating-projection algorithm used by `hdfe/reghdfe` — no 12000×12000 inversion.

“HDFE controls” = the dummy columns of ΔZ are routed through `hdfe`’s projection instead of through `matrix accum`. By (1) the $\hat{\theta}_d$ on the genuine numeric controls is unchanged.

3 The two flavors you would pass, and what each does

The first-difference structure of `did_multiplgt_dyn` would seem to rule out dummies that do not vary over time — their FD is zero. But the help file already gives the workaround:

“To control for time-invariant covariates, one can for instance input the product of those covariates and of the time variable T into the option.”

We apply that trick to both flavors of HDFE the user might want to absorb.

(A) Time-invariant dummies: `i.ac_uq_id times monthyear`

For a group-level attribute a_g that does not change over t , the bare dummy has zero first difference:

$$\Delta \mathbf{1}\{a_g = a_0\} = \mathbf{1}\{a_g = a_0\} - \mathbf{1}\{a_g = a_0\} = 0. \quad (2)$$

The fix is to multiply by time. Define $\tilde{Z}_{a_0}(g, t) := \mathbf{1}\{a_g = a_0\} \cdot t$. Then

$$\Delta \tilde{Z}_{a_0}(g, t) = \mathbf{1}\{a_g = a_0\} \cdot t - \mathbf{1}\{a_g = a_0\} \cdot (t - 1) = \mathbf{1}\{a_g = a_0\}. \quad (3)$$

The first difference is the original dummy itself — generically non-zero. So the construction is admissible as a control after all.

Interpretation. Including $\tilde{Z}_{a_0} = \mathbf{1}\{a_g = a_0\} \cdot t$ for every cluster level a_0 amounts to allowing each cluster to have its own *linear time trend* in Y . The coefficient on $\tilde{Z}_{a_0}$ is the slope of cluster a_0 ’s trend relative to the omitted category. This is the `trends_lin`-style assumption applied at the cluster level rather than the group level. With A clusters this is A extra columns, which is the column space we want `hdfe` to absorb.

(B) Cluster-by-time dummies: `i.ac_uq_id#i.monthyear times monthyear`

The same trick applies to a cluster-by-time indicator. The bare cell-dummy $\mathbf{1}\{a_g = a_0, t = t_0\}$ has a first difference that is non-zero in two adjacent periods,

$$\Delta \mathbf{1}\{a_g = a_0, t = t_0\} = \mathbf{1}\{a_g = a_0\} \cdot (\mathbf{1}\{t = t_0\} - \mathbf{1}\{t = t_0 - 1\}), \quad (4)$$

which already qualifies as a usable control. But to keep the construction *symmetric* with case (A) and to give it a linear-trend interpretation inside each (cluster, time) cell, multiply by time:

$$\tilde{Z}_{a_0, t_0}(g, t) := \mathbf{1}\{a_g = a_0, t = t_0\} \cdot t. \quad (5)$$

The first difference is then

$$\Delta \tilde{Z}_{a_0, t_0}(g, t) = \mathbf{1}\{a_g = a_0\} \cdot (t \cdot \mathbf{1}\{t = t_0\} - (t - 1) \cdot \mathbf{1}\{t = t_0 - 1\}), \quad (6)$$

which is non-zero in exactly the same two periods as (4) but weighted by t_0 and $t_0 - 1$ respectively. The two specifications span the same column space when both $\tilde{Z}_{a_0, t_0}$ for every (a_0, t_0) and the cluster-only $\tilde{Z}_{a_0}$ are included — FD-of-time-times-dummy together with FD-of-dummy together saturate “cluster-by-time” variation in linear trends.

Stata syntax for this object. Either of the following two equivalent specs:

```
* Cluster-specific linear trend + cluster x time interaction trend:
hdfe_controls( i.ac_uq_id##c.monthyear )
* Or, written out:
hdfe_controls( i.ac_uq_id i.ac_uq_id#c.monthyear )
```

The `c.monthyear` part is what implements the “times year” trick: `reghdfe` interprets `i.ac_uq_id#c.monthyear` as the interaction of each cluster dummy with the continuous time variable, giving exactly $\tilde{Z}_{a_0} = \mathbf{1}\{a_g = a_0\} \cdot \text{monthyear}$ for each a_0 . The leading `i.ac_uq_id` would be the cluster fixed effect itself; in our FD setup that piece is absorbed by group differencing automatically and can be omitted (or kept in the absorb spec where `reghdfe` will detect the collinearity without error).

Both flavors at once, in absorb syntax

When you want both case (A) and case (B) jointly, the natural absorb spec is:

```
hdfe_controls( i.ac_uq_id##c.monthyear i.ac_uq_id#i.monthyear )
```

where the first term ($\mathbf{1}\{a_g\} \cdot 1$ and $\mathbf{1}\{a_g\} \cdot t$) gives cluster fixed effects and cluster-specific linear trends, and the second term gives full cluster-by-time intercepts. The two together span the column space of any “constant for g within (a_g, t) ”-style HDFE the user might want.

This is the column space that `hdfe`’s projection is built to demean by, and the option `hdfe_controls()` simply forwards the user’s `reghdfe`-style spec into the `absorb()` argument of one `hdfe` call per baseline level.

4 Strategy: a focused option `hdfe_controls()`

```
did_multipllegt_dyn Y G T D, effects(L) [other options] ///
      controls(X1 X2)                                     ///
      hdfe_controls(ac_uq_id#monthyear)
```

Semantics. The varlist in `hdfe_controls()` is interpreted *not* as numeric controls but as a recipe for the projection P_Z^W . Each entry is a categorical variable or interaction (`reghdfe`-style absorb syntax). For each baseline level d , we partial out the joint column space of these dummies from both ΔY and each genuine numeric control ΔX_k , then estimate $\hat{\theta}_d$ on the residualized inputs.

What this option does NOT do (and why that is correct):

- **No long-difference subtraction at L4679 for the HDFE part.** L4679 subtracts $\hat{\theta}_d[k] \cdot \Delta X_{k,g,t}^{(\ell)}$ for each *numeric* control k . The HDFE projection is already taken out of the comparison through the event-study estimator’s structure: a switcher’s $\Delta Y^{(\ell)}$ is compared against not-yet-switchers *from the same control cells* via the demeaning at L4647–4664, whose denominator $\hat{E}_{g,t}^{(d)}$ is now built with the HDFE-augmented projection. The HDFE long-difference is canceled by the comparison itself, not by a separate subtraction.

- **No new variance correction $U^{\text{var},X}$ for the HDFE.** $U^{\text{var},X}$ in the companion paper captures the noise from estimating $\widehat{\theta}_d$. We do not store “one coefficient per HDFE dummy” — we treat the projection as a nuisance subspace, the same way `trends_nonparam` does. The asymptotics are the `trends_nonparam` asymptotics with a richer projector, which Section 1.4 of dCdH (2024) covers verbatim for `stratum`×`time` structures.

5 Why this is the right answer mathematically

Two facts make the fast path identical to “include them as controls”:

Fact 1 (FWL). By (1), $\widehat{\theta}_d$ on the genuine numeric controls is the same whether you (i) include ΔZ as columns of the design matrix or (ii) partial out ΔZ from both sides.

Fact 2 (Variance). In the dCdH (2024) framework, the asymptotic variance with K *finite* numeric controls has the $U^{\text{var},X}$ correction. With $K \rightarrow \infty$ (the HDFE limit), the asymptotic framework treats the FE projection as a deterministic nuisance subspace and **does not** add a K -growing variance correction — that is the correct asymptotic posture for the absorbed dimensions. So when we route 12,000 dummies through `hdfe`, we are simultaneously (a) computing the right point estimate by FWL and (b) using the right asymptotic variance by the FE-projection asymptotics.

The `trends_nonparam` option in the file is exactly this regime applied to one stratum. We are generalizing to many.

6 What changes in the .ado, by stage

Stage 1 — the projection enlarges

The projector that residualizes ΔY and each ΔX_k becomes:

$$M_{\mathcal{F}_d}^W, \quad \mathcal{F}_d = \text{span}\{\mathbb{1}\{t = t_0\}, \mathbb{1}\{s_g = s_0\} \cdot \mathbb{1}\{t = t_0\}, Z_{g,t}^{(p)}\}, \quad (7)$$

where $Z^{(p)}$ ranges over the absorb terms in `hdfe_controls()`. Within each baseline cell d , one `hdfe` call demeans ΔY and all genuine ΔX_k by $\mathcal{F}_d$ in a single sweep.

Code touches. The four substitutions are the same as in the “cluster_time_fe” plan, but for a different conceptual reason:

- L941: $\widetilde{\Delta X}_k = \sqrt{N_{g,t}} \cdot [M_{\mathcal{F}_d}^W \Delta X_k]$.
- L955: $\widetilde{\Delta Y} = \sqrt{N_{g,t}} \cdot [M_{\mathcal{F}_d}^W \Delta Y]$.
- L958: $\text{prod_}X_{k,g,t} = N_{g,t} \cdot [M_{\mathcal{F}_d}^W \Delta X_k]$.
- L984–996: $\widehat{E}_{g,t}^{(d)} = [P_{\mathcal{F}_d}^W \Delta Y]$ via `reghdfe diff_y_XX 'controlsXX', absorb('hdfe_set') residuals(...)`.

Stages 2 and 3 — unchanged

Stages 2 and 3 read `coefs_sq_‘d’_XX` (numeric-controls slope only), `inv.Denom_‘d’_XX` (built off the augmented projection), `prod.X*Ngt_XX` (per numeric control only), and `E_y_hat_gt_int_‘d’_XX` (now includes the HDFE part). All four are correctly rebuilt by the Stage-1 patch. The long-difference residualization at L4679 and the variance correction at L4905–4954 then run verbatim and produce the right event-study estimates and SEs.

7 Performance estimate

Approach	Storage	Time per baseline d	Numerical
<code>controls(12000 dummies)</code>	12000 N cells + $144 \cdot 10^6$ matrix	$O(K^3) = 10^{12}$ flops	singular decomposition
<code>hdfc_controls(ac.uq.id#monthyear)</code>	N cells per demeaned variable	$O(N \cdot \text{iters})$, $\text{iters} \leq 50$	none (alternating)

On a panel with $N = 200,000$, $K = 12,000$, and 4–5 baseline levels, the naive path is unrunnable in reasonable time on standard hardware. The HDFS path is seconds: `hdfc` typically converges in fewer than 30 iterations, each linear in N .

8 Stata patch

(A) Syntax + validation [\[ado L53 / 65–73\]](#)

```
* L53 -- add the option (note the string type to accept reghdfe absorb syntax):
syntax varlist(min=4 max=4 numeric) [if] [in] [, /* ... */ hdfc_controls(string)]

* After the gtools check at L65-73:
if "`hdfc_controls'" != "" {
    qui cap which reghdfe
    if _rc { di as error "hdfc_controls() requires reghdfe. Run: ssc install reghdfe" ;
    exit 198 }
    qui cap which hdfc
    if _rc { di as error "hdfc_controls() requires hdfc. Run: ssc install hdfc" ; exit
    198 }

    * Warn if the user passes a purely time-invariant variable (no-op).
    foreach a of local hdfc_controls {
        capture confirm variable `a'
        if !_rc {
            tempvar sd_a
            bys `2': egen `sd_a' = sd(`a')
            sum `sd_a'
            if r(max) == 0 {
                di as text "Note: `a' is time-invariant within group; it is absorbed by
" ///
                "differencing and will not contribute. Ignoring."
                local hdfc_controls : subinstr local hdfc_controls "`a'" "", word
            }
            drop `sd_a'
        }
    }
}
```

```

    }
  }
}

```

(B) Stage 1 — demean + theta + Den_d (replaces L919–1098)

```

* Build the absorb spec: time FE, optional trends_nonparam x time, plus user's
hdfe_controls.
local fe_set "time_XX"
if "'trends_nonparam'" != "" {
    capture drop trends_nonparam_temp_XX
    gegen trends_nonparam_temp_XX = group('trends_nonparam')
    local fe_set "'fe_set' i.trends_nonparam_temp_XX#i.time_XX"
}
if "'hdfe_controls'" != "" {
    local fe_set "'fe_set' 'hdfe_controls'"
}

* Allocate the residual containers used downstream.
local mycontrols_XX ""
local count_controls = 0
foreach var of varlist 'controls' {
    local ++count_controls
    cap drop resid_X'count_controls'_time_FE_XX
    cap drop prod_X'count_controls'_Ngt_XX
    gen resid_X'count_controls'_time_FE_XX = 0
    gen prod_X'count_controls'_Ngt_XX = 0
    local mycontrols_XX "'mycontrols_XX' resid_X'count_controls'_time_FE_XX"
}
cap drop diff_y_wXX
gen diff_y_wXX = 0

levelsof d_sq_int_XX, local(levels_d_sq_XX)

foreach l of local levels_d_sq_XX {

    * Control sample for baseline d = 'l'.
    tempvar smp
    gen byte 'smp' = (ever_change_d_XX == 0          ///
                     & diff_y_XX != .                ///
                     & fd_X_all_non_missing_XX==1    ///
                     & d_sq_int_XX == 'l'            ///
                     & time_XX < F_g_XX)

    * One hdfe call: demean DY and all numeric DX_k by F_d simultaneously.
    local dvars "diff_y_XX"
    forvalues k = 1/'count_controls' { local dvars "'dvars' diff_X'k'_XX" }

    qui hdfe 'dvars' [aw=N_gt_XX] if 'smp', ///
        absorb('fe_set') generate(_resXX_)

    * Pack residuals into the names the unchanged tail expects.
    replace diff_y_wXX = sqrt(N_gt_XX) * _resXX_diff_y_XX ///

```

```

    if d_sq_int_XX == 'l' & 'smp' == 1
forvalues k = 1/'count_controls' {
    replace resid_X'k'_time_FE_XX = sqrt(N_gt_XX) * _resXX_diff_X'k'_XX ///
        if d_sq_int_XX == 'l' & 'smp' == 1
    replace prod_X'k'_Ngt_XX = sqrt(N_gt_XX) * resid_X'k'_time_FE_XX ///
        if d_sq_int_XX == 'l'
}
drop _resXX_*

* --- E_y_hat_gt_int_l_XX: the full fitted value used downstream at L4664 ---
cap drop E_y_hat_gt_int_'l'_XX
local numericsXX ""
forvalues k = 1/'count_controls' { local numericsXX "'numericsXX' diff_X'k'_XX" }

cap reghdfe diff_y_XX 'numericsXX' [aw=N_gt_XX] ///
    if d_sq_int_XX == 'l' & time_XX < F_g_XX, ///
    absorb('fe_set') noconstant residuals(_res_DY_l_XX)
scalar rc_XX = _rc
if rc_XX == 0 {
    gen E_y_hat_gt_int_'l'_XX = diff_y_XX - _res_DY_l_XX ///
        if d_sq_int_XX == 'l' & time_XX < F_g_XX
    drop _res_DY_l_XX
    scalar df_a_'l'_XX = e(df_a)
}
else {
    drop if d_sq_int_XX == 'l'
    noi di "Baseline 'l' dropped because of insufficient observations."
}

drop 'smp'

* The tail of the existing block (L1018-1098) runs verbatim from here:
* matrix accum overall_XX = diff_y_wXX 'mycontrols_XX' if d_sq_int_XX=='l' & ...
* matrix coefs_sq_'l'_XX = invsym(didmgt_XX)*didmgt_Xy
* matrix inv_Denom_'l'_XX = invsym(didmgt_XX) * r(sum) * G_XX
* etc.
}

```

The key efficiency point: `count_controls` above is the number of *genuine numeric controls only* (e.g. 2), not 12,000. The 12,000 dummies enter solely through the `absorb('fe_set')` argument, which `hdfe` handles with $O(N)$ -per-iteration alternating projections instead of an $O(K^3)$ matrix inversion.

(C) Stages 2 and 3 — NO changes

The blocks at L4582–4689 (long-difference residualization for effects), L5145–5184 (placebos), and L4905–4954 (variance correction) compile and run as written. They consume the Stage-1 outputs which now incorporate the HDFE projection.

9 End-to-end example

```
import delimited using "data_test.csv", clear
```

```

* Numeric controls: X1 X2.
* HDFE: ac_uq_id (time-invariant -> auto-dropped) and ac_uq_id#monthyear.
did_multipligt_dyn Y unique_small_grid_id monthyear D, ///
    effects(4) placebo(2)                ///
    controls(X1 X2)                      ///
    hdfc_controls(ac_uq_id#monthyear)    ///
    cluster(ac_uq_id)

```

What is being estimated, per baseline d :

$$\hat{\theta}_d = \arg \min_{\theta, \alpha^{(d)}} \sum_{(g,t) \in \mathcal{S}_d} N_{g,t} \left(\Delta Y_{g,t} - \Delta X_{g,t}^\top \theta - \alpha_{a_g,t}^{(d)} \right)^2, \quad (8)$$

which is identical to what `controls(X1 X2 i.ac_uq_id#i.monthyear)` *would* compute, except that the 12,000-dimensional $\alpha^{(d)}$ is absorbed instead of materialized.

10 Sanity checks

1. **Single-attribute equivalence.** `hdfc_controls(i.ac_uq_id#i.monthyear)` should give the same coefficients and SEs as `trends_nonparam(ac_uq_id)`, because both produce the same projection $P_{\mathcal{F}_d}^W$ on `data_test.csv`.
2. **Time-invariant drop.** `hdfc_controls(i.ac_uq_id)` should print a notice “time-invariant within group, ignoring” and run as if no HDFE were passed. The point estimates and SEs should match the no-controls-FE command.
3. **FWL self-check.** On a small sub-sample where 12,000 dummies actually fit in memory, `controls(X1 X2 d1 d2 ... d12000)` and `hdfc_controls(i.ac_uq_id#i.monthyear)` should give identical $\hat{\theta}_d$ on (X1, X2) up to floating-point precision.

11 Summary

The user wants 12,000+ `ac_uq_id#monthyear` dummies as controls. Including them via `controls()` is computationally infeasible ($O(K^3)$ inverse), and time-invariant dummies inside that set are killed by differencing anyway. The fix is a new option `hdfc_controls(spec)` accepting `reghdfe`-style absorb syntax. It enlarges Stage 1’s projector $\mathcal{F}_d$ with the user’s HDFE dimensions, uses one `hdfc` call per baseline level to demean ΔY and the genuine numeric controls, then runs the existing `matrix accum`, $\hat{\theta}_d$, and Den_d^{-1} code unchanged. Stages 2 and 3 (L4582–4689 and L4905–4954) need no edits: they consume the rebuilt Stage-1 outputs. By Frisch–Waugh–Lovell the point estimates match the limit of “infinitely many dummy controls,” and the variance follows the `trends_nonparam`-style nuisance-projection asymptotics of dCdH (2024) Sec. 1.4.